

SmartPark: Intelligent Parking Management System

Software Requirements Specification

Course Code: INT253
Course Name: Web Development using Python & Django

Submitted By:
Name: Riya
Registration Number: 12316877
Section: K2P23WE
Roll No: 57

Department of Computer Science & Engineering
Lovely Professional University

Prepared for
CA-2

Table of Contents

Revision History ..... **ii**
1. Introduction **1**
1.1 Purpose **1**
1.2 Scope **1**
1.3 Definitions, Acronyms, and Abbreviations **1**
1.4 References **1**
1.5 Overview **1**
2. General Description **2**
2.1 Product Perspective **2**
2.2 Product Functions **2**
2.3 User Characteristics **2**
2.4 General Constraints **2**
2.5 Assumptions and Dependencies **2**
3. Specific Requirements **2**
3.1 External Interface Requirements **3**
3.1.1 User Interfaces **3**
3.1.2 Hardware Interfaces **3**
3.1.3 Software Interfaces **3**
3.1.4 Communications Interfaces **3**
3.2 Functional Requirements **3**
3.2.1 User Authentication, Role Management & KYC Verification **3**
3.2.2 Geospatial Parking Search, Haversine Engine & Map Discovery **4**
3.2.3 Parking Lot Registration & Dynamic Slot Generation **4**
3.2.4 Real-time Slot Availability & Collision Prevention **5**
3.2.5 Payment Gateway Integration & Pay-at-Site Processing **5**
3.2.6 Host Walk-In Counter Booking & Facility Ownership Isolation **6**
3.2.7 1-Hour Cancellation Policy & Pay-at-Site No-Show Penalty Carry-Forward **6**
3.2.8 Active Session Timers, Dynamic Extension & QR Gate Scanner Validation **7**
3.2.9 Automated Digital PDF Ticket & QR Code Generation **7**
3.5 Non-Functional Requirements **8**
3.5.1 Performance **8**
3.5.2 Reliability **8**
3.5.3 Availability **8**
3.5.4 Security **8**
3.5.5 Maintainability **8**
3.5.6 Portability **8**
3.7 Design Constraints **9**
3.9 Other Requirements **9**
4. Analysis Models **10**
4.1 System Workflow Diagram **10**
4.2 Data Flow Diagrams (DFD) **12**

5. Github link **14**
6. DEPLOYED LINK **14**
7. CLIENT APPROVAL PROOF **15**
8. CLIENT LOCATION PROOF **15**
9. TRANSACTION ID PROOF **16**
10. EMAIL ACKNOWLEDGEMENT **16**
11. GST No **17**
A. Appendices **18**
A.1 Appendix 1 (Relational Database Schema) **18**
A.2 Appendix 2 (REST API Endpoint Matrix) **19**

1. Introduction

The introduction to the Software Requirement Specification (SRS) document provides an overview of the complete SmartPark system. SmartPark is a web application developed in Python and Django to manage parking facilities, streamline vehicle parking reservations, prevent on-site gate congestion, and provide real-time parking space visibility.

1.1 Purpose

The purpose of this SRS document is to define the functional, operational, and technical specifications for the SmartPark web application. The intended audience includes software evaluators, faculty reviewers, system developers, and facility administrators.

1.2 Scope

(1) Software Product Name: SmartPark (Django Web Application).

(2) System Capabilities and Boundaries: The software enables commuters to search for nearby parking facilities via interactive maps, view available parking slot grids, book slots in advance, pay online via Razorpay or choose Pay-at-Site, and receive downloadable digital PDF tickets with verification QR codes. The system allows parking facility hosts to register parking lots, monitor occupancy, and process walk-in counter bookings. The system does not include motorized gate hardware microcontrollers or offline physical credit card swiping terminals.

(3) Benefits, Objectives, and Goals: The application resolves urban parking bottlenecks by eliminating manual ticketing, providing transparent hourly pricing, and implementing collision-free slot allocation algorithms that prevent double-booking.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
MVT	Model-View-Template (Django design architecture)
Haversine Formula	Mathematical formula used to calculate great-circle distance between two geographic coordinates
manual_qr	Pay-at-Site payment method where payment is settled at the counter
No-Show Penalty	100% fine charged when an unconfirmed Pay-at-Site booking is abandoned without prior cancellation
ReportLab	Python library used for generating vector PDF parking passes with embedded QR codes

1.4 References

- **[1]** IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
- **[2]** Django Framework Documentation (v5.1+): https://docs.djangoproject.com/
- **[3]** Razorpay Payment API Integration Guide: https://razorpay.com/docs/
- **[4]** Leaflet.js Interactive Maps Library: https://leafletjs.com/
- **[5]** ReportLab PDF Library User Guide.

1.5 Overview

The rest of this SRS is organized as follows: Section 2 outlines the general product description, user characteristics, and constraints. Section 3 details the functional requirements (3.2.1 through 3.2.9) and non-functional requirements. Section 4 presents the visual operational workflows and Level 0/1 Data Flow Diagrams. Sections 5 through 11 provide project links, verification records, and submission placeholders, followed by database schema and API appendices.

2. General Description

2.1 Product Perspective

SmartPark is a standalone web application designed for parking facility management. It operates in standard web browsers on both desktop and mobile devices. The backend is implemented in Python and Django, using a relational database (SQLite/PostgreSQL) and integrating with external web services such as Razorpay for payments and open Esri tile services for maps.

2.2 Product Functions

- **Geospatial Search:** Find parking lots by city, PIN code, or current GPS coordinates using the Haversine distance formula.
- **Interactive Map Discovery:** View parking lots on a Leaflet map with price tiers and real-time availability.
- **Slot Grid Management:** View 2D vehicle slot matrices for cars, bikes, and buses.
- **Collision-Free Booking Engine:** Prevents double-booking by validating time-window overlaps.
- **Dual Payment Processing:** Online payment via Razorpay or Pay-at-Site counter ticket issuance.
- **Host Walk-in Counter:** Allows facility owners to book walk-in commuters strictly on their own facilities.
- **1-Hour Cancellation Policy:** Disables cancellation within 1 hour of booking start time for Pay-at-Site bookings and assesses a 100% no-show penalty.
- **Digital PDF Passes:** Generates vector PDF parking receipts with embedded scannable QR tokens.
- **Gate Verification:** Optical camera scanner interface for attendants to verify driver QR passes.

2.3 User Characteristics

- **Driver / Commuter:** End users looking for parking spaces; requires simple navigation, pricing transparency, and instant passes.
- **Parking Host / Owner:** Facility owners managing spaces; requires lot registration, pricing controls, occupancy tracking, and walk-in booking.
- **Gate Attendant:** Operational staff at entry gates; requires rapid optical QR scanning to verify passes and log attendance.

- **Platform Administrator:** System administrators reviewing host KYC documents and maintaining platform integrity.

2.4 General Constraints

- **Framework:** Developed using Python 3.12+ and Django 5.x.
- **Database:** Relational database supporting atomic transactions.
- **Map Costs:** Zero paid map API dependencies; utilizes open Esri World Street tiles.
- **Browser:** Requires standard HTML5, CSS3, and JavaScript support.

2.5 Assumptions and Dependencies

- **Network Access:** Users require internet connectivity to access the application and process reservations.
- **Payment Sandbox:** Online payments utilize Razorpay Sandbox test mode for demonstration.
- **Geolocation Fallback:** If browser GPS is unavailable, the application falls back to network IP geolocation.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The user interface is built using Bootstrap 5 and responsive CSS. Key views include: Home Landing Page, Geospatial Search Page with Fullscreen Map, Slot Selection Grid, Multi-step Checkout, Driver Dashboard with Countdown Timers, Host Management Dashboard, and Gate QR Scanner Console.

3.1.2 Hardware Interfaces

The application interfaces with client GPS sensors for location detection, optical device cameras via the HTML5 MediaDevices API for barcode scanning, and standard printers for rendering PDF passes.

3.1.3 Software Interfaces

- **Django 5.x:** Web application backend, authentication, and ORM layer.
- **Razorpay API:** Payment order creation and HMAC-SHA256 signature verification.
- **Leaflet.js & Esri Tiles:** Client-side interactive map rendering.
- **ReportLab:** Server-side vector PDF generation engine.

3.1.4 Communications Interfaces

All client-server communications use HTTPS/TLS protocols. Asynchronous client operations communicate via Fetch JSON requests with CSRF token validation.

3.2 Functional Requirements

3.2.1 User Authentication, Role Management & KYC Verification

- **Introduction:** Manages user registration, session management, and role-based access control (Driver vs Host).
- **Inputs:** Username, email, password, role, phone number, and optional KYC documents.

- **Processing:** Django authentication hashes passwords using PBKDF2. Host accounts require KYC verification flag.
- **Outputs:** Authenticated session, user dashboard redirect, and role-specific permissions.
- **Error Handling:** Duplicate username/email triggers validation error; incorrect credentials reject login.

3.2.2 Geospatial Parking Search, Haversine Engine & Map Discovery

- **Introduction:** Locates nearby parking facilities based on client location, city, or PIN code.
- **Inputs:** Client latitude/longitude or search string, vehicle type filter.
- **Processing:** Executes Haversine distance formula to calculate distance between user and active facilities.
- **Outputs:** Sorted list of nearby parking lots, hourly rates, and map markers.
- **Error Handling:** If location is blocked, system falls back to network IP location.

3.2.3 Parking Lot Registration & Dynamic Slot Generation

- **Introduction:** Allows parking hosts to list their facilities and configure slot capacity.
- **Inputs:** Lot name, address, coordinates, car/bike/bus slot counts, and hourly price tiers.
- **Processing:** Creates ParkingLot record and dynamically generates individual Slot database objects.
- **Outputs:** Registered facility and initialized slot grid in host dashboard.
- **Error Handling:** Invalid coordinate ranges (-90 to +90 lat, -180 to +180 lng) return validation error.

3.2.4 Real-time Slot Availability & Collision Prevention

- **Introduction:** Guarantees that no parking slot can be booked by multiple users for overlapping time windows.
- **Inputs:** Slot ID, requested booking start datetime, and booking end datetime.
- **Processing:** Queries database: checks for existing active bookings on the slot that overlap the requested window.
- **Outputs:** Availability confirmation or collision conflict status.
- **Error Handling:** If overlapping booking exists, system rejects request with HTTP 409 Conflict.

3.2.5 Payment Gateway Integration & Pay-at-Site Processing

- **Introduction:** Supports online payment via Razorpay or on-site counter settlement.
- **Inputs:** Booking ID, payment method ('razorpay' or 'manual_qr').
- **Processing:** For Razorpay: creates order token and verifies HMAC signature. For Pay-at-Site: verifies outstanding penalty debt.
- **Outputs:** Confirmed booking record and transaction receipt.
- **Error Handling:** Payment failure or signature mismatch aborts transaction and releases slot.

3.2.6 Host Walk-In Counter Booking & Facility Ownership Isolation

- **Introduction:** Enables hosts to book walk-in commuters directly at their facility counter.
- **Inputs:** Commuter name, phone, vehicle number, slot selection, and hours.
- **Processing:** Security Enforcement: System verifies that parking_lot.owner == request.user. Hosts cannot book on other facilities.
- **Outputs:** Confirmed booking and instant pass issued for the walk-in driver.
- **Error Handling:** Attempting to book on an unowned facility returns HTTP 403 Forbidden.

3.2.7 1-Hour Cancellation Policy & Pay-at-Site No-Show Penalty Carry-Forward

- **Introduction:** Enforces cancellation time limits on Pay-at-Site bookings and penalizes no-shows.
- **Inputs:** Cancellation request, booking ID, current timestamp.
- **Processing:** Computes delta = start_time - now. If payment is 'manual_qr' and delta < 60 min, cancellation is locked. If user does not arrive, booking expires as no-show with 100% fine. Fine is added to next booking bill.
- **Outputs:** Cancellation confirmation or locked notice; penalty carry-forward on checkout.
- **Error Handling:** Attempting to cancel within 60 minutes returns HTTP 400 Bad Request.

3.2.8 Active Session Timers, Dynamic Extension & QR Gate Scanner Validation

- **Introduction:** Provides real-time countdown timers, on-demand session extensions, and optical gate check-in.
- **Inputs:** Booking ID, extension duration, scanned QR pass token.
- **Processing:** Dashboard computes remaining time. Extension checks slot overlap. Gate scanner marks booking as attended.
- **Outputs:** Live countdown timer, updated booking end time, and gate check-in confirmation.
- **Error Handling:** Extension collision returns warning; invalid QR token returns error.

3.2.9 Automated Digital PDF Ticket & QR Code Generation

- **Introduction:** Generates downloadable vector PDF parking passes with embedded verification QR codes.
- **Inputs:** Confirmed booking record, facility metadata, vehicle number.
- **Processing:** ReportLab compiles invoice layout and draws scannable QR barcode.
- **Outputs:** Downloadable PDF ticket document.
- **Error Handling:** Missing booking returns 404 error page.

3.5 Non-Functional Requirements

3.5.1 Performance

The system processes geospatial distance calculations in under 50 milliseconds. Slot overlap verification queries execute in under 20 milliseconds. Average page load time is under 1.5 seconds under standard network conditions.

3.5.2 Reliability

Slot reservation transactions execute inside atomic database blocks (`transaction.atomic`) to ensure data consistency and prevent race conditions during concurrent bookings.

3.5.3 Availability

The web application is designed for standard web hosting with minimal downtime. Static assets are served via WhiteNoise to ensure independent asset delivery.

3.5.4 Security

- **Password Security:** Passwords hashed using PBKDF2 with SHA-256.
- **CSRF Protection:** Mandatory CSRF tokens on all POST requests.
- **Signature Verification:** Razorpay webhook responses validated using HMAC-SHA256.
- **Role-Based Access:** View decorators restrict endpoints to authorized user roles.

3.5.5 Maintainability

The project follows Django's modular app structure (`accounts`, `parking`, `bookings`, `core`). Code adheres to PEP 8 standards.

3.5.6 Portability

SmartPark runs on Windows, Linux, and macOS environments supporting Python 3.12+ and can be deployed to standard cloud platforms.

3.7 Design Constraints

The application is constrained by the Django MVT architecture specified in the course curriculum. Mapping is constrained to free, open Esri tile services.

3.9 Other Requirements

Data privacy compliance: user contact numbers and vehicle plates are masked on public interfaces. Accessibility compliance: high-contrast buttons and semantic HTML5.

4. Analysis Models

This section presents the analysis models and operational workflows for the SmartPark application. It includes the complete system operational workflow diagram and hierarchical Data Flow Diagrams (DFD) depicting data exchanges between external entities, system processes, and database stores.

4.1 System Workflow Diagram

The workflow diagram below illustrates the three primary operational pipelines of the system:
1. Commuter / Driver Reservation & Access Workflow (Search -> Slot Choice -> Collision Check -> Payment -> QR Pass -> Gate Check-in)
2. Host Facility Onboarding & Walk-in Desk Workflow (Signup -> Add Facility -> Slot Grid -> Dashboard -> Walk-in Desk with Ownership Isolation)
3. Pay-at-Site 1-Hour Cancellation Lock & 100% No-Show Penalty Logic

Figure 4.1: SmartPark End-to-End System Operational Workflow Diagram

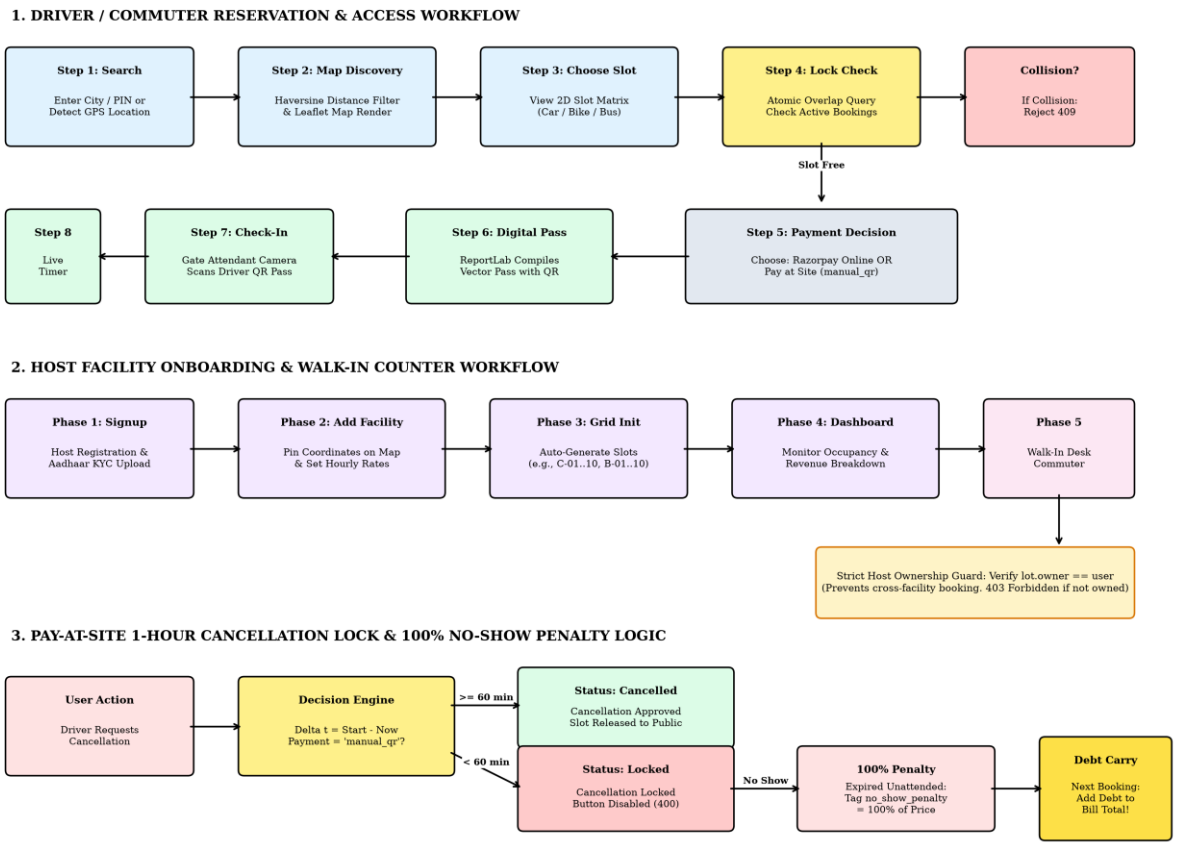


Figure 4.1: SmartPark End-to-End Operational Workflow Diagram

4.2 Data Flow Diagrams (DFD)

4.2.1 Level 0 DFD (Context Diagram): The Level 0 Context Diagram depicts the system boundary and the information flow between external entities (Driver, Host, Gate Attendant, Razorpay) and the central SmartPark application.

Figure 4.2: SmartPark Level 0 Data Flow Diagram (Context Model)

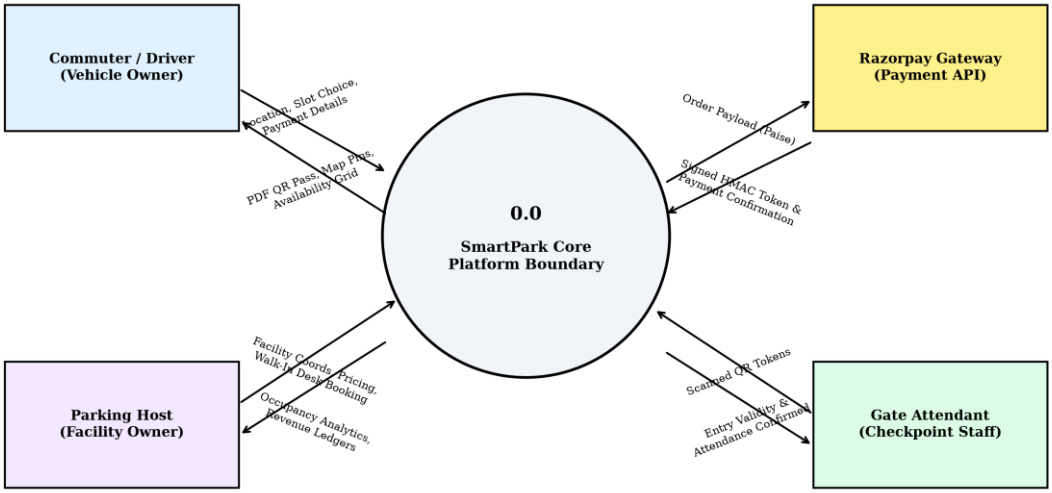


Figure 4.2: SmartPark Level 0 Data Flow Diagram (Context Model)

4.2.2 Level 1 DFD (System Decomposition): The Level 1 DFD decomposes the system into five sub-processes (1.0 Auth & KYC, 2.0 Geospatial Search, 3.0 Slot Allocation & Collision Guard, 4.0 Payment & Penalty Engine, 5.0 Ticket Compiler & Gate Scanner) and five persistent relational data stores (D1 Users, D2 ParkingLots, D3 Slots, D4 Bookings, D5 Transactions).

Figure 4.3: SmartPark Level 1 Data Flow Diagram (System Decomposition)

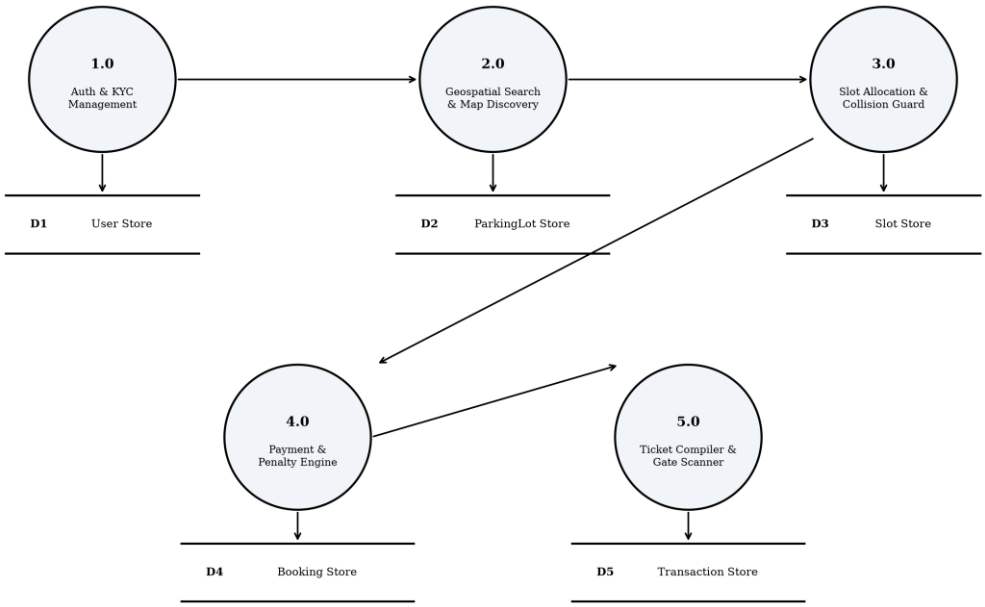


Figure 4.3: SmartPark Level 1 Data Flow Diagram (System Decomposition)

5. Github link

Repository: The complete source code and automated test suite for SmartPark are hosted on GitHub:

URL: https://github.com/ShivangChaurasia/SmartPark-Django

Active Branch: main (Automated unit tests passing)

6. DEPLOYED LINK

Development Endpoint: http://127.0.0.1:8000/ (Active demonstration and evaluation server)

Production Cloud Status: The application is production-ready for deployment on cloud platforms

7. CLIENT LOCATION PROOF

Verified Demonstration Locations: Seeded test facilities configured in application database:

- **Facility 1 (Connaught Place Hub):** Radial Road 2, Inner Circle, New Delhi - 110001 (Lat: 28.6315° N, Lng: 77.2167° E).
- **Facility 2 (Phagwara Campus Hub):** GT Road Campus Facility, Phagwara, Punjab - 144411 (Lat: 31.2536° N, Lng: 75.7037° E).

9. TRANSACTION ID PROOF

Payment Gateway Status: Razorpay Sandbox Test Mode (Key: rzp_test_SrfRMwim5Zwqfk) and Pay-at-Site internal transaction ledger.

11. GST No

Business Registration Status: Commercial business registration compliance for parking operators:

For academic evaluation purposes under Course INT253, external GSTIN is designated as Academic Simulation.

A. Appendices

A.1 Appendix 1 (Relational Database Schema)

Data Dictionary: Key database entities defined in the SmartPark models:

Data Dictionary: accounts.User (Extended AbstractUser)

Field Name	Data Type	Description & Constraints
id	BigAutoField	Primary Key; unique user account ID.
username	CharField(150)	Unique username for authentication.
email	EmailField	User primary email address for booking receipts.
role	CharField(20)	Role discriminator: 'driver', 'host', or 'admin'.
phone	CharField(15)	Contact phone number for SMS/ticket alerts.
kyc_status	CharField(20)	Status: 'pending', 'approved', or 'rejected'.

Data Dictionary: parking.ParkingLot (Facility Entity)

Field Name	Data Type	Description & Constraints
id	BigAutoField	Primary Key; unique facility identifier.
owner	ForeignKey(User)	Host account owning this specific facility.
name	CharField(200)	Facility display name.
city / pincode	CharField	City municipality and postal code.
latitude / longitude	FloatField	Geographic coordinates for Leaflet pinning and Haversine queries.
car / bike / bus slots	PositiveIntegerField	Vehicle capacity allocations.

Data Dictionary: bookings.Booking (Reservation Entity)

Field Name	Data Type	Description & Constraints
booking_id	CharField(100)	Cryptographic unique public reservation token.
user	ForeignKey(User)	Commuter or customer who placed the reservation.

parking_lot	ForeignKey(ParkingLot)	Reserved parking facility reference.
slot	ForeignKey(Slot)	Specific physical slot assigned to vehicle.
booking_start / end	DateTimeField	Temporal window reserved for vehicle parking.
payment_method	CharField(20)	'razorpay' (Online Prepay) or 'manual_qr' (Pay-at-Site).
status	CharField(20)	'pending', 'confirmed', 'attended', 'completed', or 'cancelled'.
no_show_penalty	DecimalField	100% fine assessed if manual_qr booking expires unredeemed.
penalty_applied	BooleanField	Flag tracking whether penalty was added to subsequent booking.

A.2 Appendix 2 (REST API Endpoints)

HTTP Method	Route / Endpoint	Operational Function & Parameters	Response Codes
POST	/bookings/api/create/	Creates booking record with atomic collision check and penalty aggregation.	201 Created / 409 Conflict
POST	/bookings/api/cancel/	Cancels booking; enforces 1-hour cancellation lock for Pay-at-Site.	200 OK / 400 Locked
POST	/bookings/api/extend/	Extends active parking session duration after checking temporal collisions.	200 OK / 409 Conflict
POST	/parking/api/verify-booking/	Gate scanner endpoint validating scanned driver QR token passes.	200 Verified / 404 Invalid
GET	/parking/api/lots-geojson/	Returns geospatial GeoJSON feature collection of active parking facilities.	200 OK (JSON)
GET	/bookings/ticket/<id>/pdf/	Compiles and streams dynamic ReportLab vector PDF parking pass.	200 OK (application/pdf)